Question 1:

An organization uses an Apache web server on a Linux platform to host its internal applications. During a routine security audit, you are tasked with identifying and mitigating potential vulnerabilities associated with the server's configuration.

Tasks:

1. **Identify** three common misconfigurations in Apache web servers that could lead to security vulnerabilities.
2. **Describe** the potential risks associated with each misconfiguration.
3. **Provide** the CLI commands or configuration file edits necessary to remediate each misconfiguration.

Solution:

1. **Misconfiguration: Directory Listing Enabled**
   - **Risk:** When directory listing is enabled, attackers can view the contents of directories that lack an index file, potentially exposing sensitive files and information.
   - **Remediation:**
     To disable directory listing, edit the Apache configuration file (typically located at `/etc/apache2/apache2.conf` or `/etc/httpd/conf/httpd.conf`) and modify the `<Directory>` directive:
     ```
     <Directory /var/www/html>
         Options -Indexes
     </Directory>
     ```
     Then, restart Apache to apply the changes:
     ```
     sudo systemctl restart apache2  # For Debian/Ubuntu systems
     sudo systemctl restart httpd    # For Red Hat/CentOS systems
     ```
2. **Misconfiguration: Server Signature and Server Tokens Exposed**
   - **Risk:** Exposing server version information can aid attackers in identifying specific vulnerabilities associated with the disclosed software versions.
   - **Remediation:**
     To hide server version details, add or modify the following directives in the Apache configuration file:
     ```
     ServerSignature Off
     ServerTokens Prod
     ```
     Apply the changes by restarting Apache:
     ```
     sudo systemctl restart apache2  # For Debian/Ubuntu systems
     sudo systemctl restart httpd    # For Red Hat/CentOS systems
     ```
3. **Misconfiguration: Unnecessary Modules Enabled**
   - **Risk:** Enabled but unused modules increase the attack surface, potentially introducing vulnerabilities that could be exploited by attackers.
   - **Remediation:**
     First, identify the enabled modules:
     ```
     apache2ctl -M  # For Debian/Ubuntu systems
     httpd -M       # For Red Hat/CentOS systems
     ```
     Then, disable unnecessary modules by commenting them out in the Apache configuration files or using the `a2dismod` command (Debian/Ubuntu):
     ```
     sudo a2dismod module_name  # Replace 'module_name' with the actual module name
     ```
     For Red Hat/CentOS systems, edit the configuration file and comment out or remove the `LoadModule` directive corresponding to the module. After making changes, restart Apache:
     ```
     sudo systemctl restart apache2  # For Debian/Ubuntu systems
     sudo systemctl restart httpd    # For Red Hat/CentOS systems
     ```

Question 2:

During a security assessment of an Apache web server running on a Linux system, you discover that the server is vulnerable to the Shellshock bug due to an outdated Bash version.

Tasks:

1. **Explain** the nature of the Shellshock vulnerability and how it can affect an Apache web server.
2. **Demonstrate** how to test if the current Bash version is vulnerable using the CLI.
3. **Outline** the steps to patch the vulnerability on a Linux system.

Solution:

1. **Nature of the Shellshock Vulnerability:**
   Shellshock is a security bug in the Bash shell that allows attackers to execute arbitrary commands on a vulnerable system by exploiting improperly sanitized environment variables. In the context of an Apache web server, if CGI scripts are enabled and invoke Bash, an attacker can craft malicious requests that execute unauthorized commands on the server, potentially leading to full system compromise. Wikipedia
2. **Testing for Vulnerability:**
   To check if the installed version of Bash is vulnerable, execute the following command in the terminal:
   `env x='() { :;}; echo Vulnerable' bash -c "echo This is a test"`
   If the output includes the word "Vulnerable," the Bash version is susceptible to Shellshock.
3. **Patching the Vulnerability:**
   To update Bash and mitigate the Shellshock vulnerability:
   - **For Debian/Ubuntu Systems:**
     ```
     sudo apt-get update
     sudo apt-get install --only-upgrade bash
     ```
   - **For Red Hat/CentOS Systems:**
     ```
     sudo yum update bash
     ```

   After updating, re-run the vulnerability test to ensure the issue has been resolved.

Question 3:

An Apache web server on a Linux platform has been identified as vulnerable to the Heartbleed bug due to an outdated OpenSSL library.

Tasks:

1. **Describe** the Heartbleed vulnerability and its potential impact on web servers.
2. **Provide** the CLI commands to determine the current version of OpenSSL installed on the server.
3. **Detail** the steps required to update OpenSSL and ensure the Apache server utilizes the patched version.

Solution:

1. **Description of the Heartbleed Vulnerability:**
   Heartbleed is a severe security vulnerability in OpenSSL's implementation of the TLS/DTLS heartbeat extension. It allows an attacker to read memory from the web server, potentially exposing sensitive data such as SSL/TLS private keys, user credentials, and other confidential information. This can lead to unauthorized access, session hijacking, and even full compromise of encrypted communications.
2. **Checking the Current OpenSSL Version:**
   To determine if the server is running a vulnerable version of OpenSSL, execute the following command:
   `openssl version -a`
   Vulnerable versions include OpenSSL 1.0.1 through 1.0.1f. If the installed version falls within this range, it must be updated immediately.
   Additionally, to check if the system is affected by Heartbleed, use the `nmap` script (if `nmap` is installed):
   `nmap --script ssl-heartbleed -p 443 <server_ip>`
   If the output confirms vulnerability, proceed with mitigation.
3. **Updating OpenSSL and Restarting Apache:**
   For Debian/Ubuntu Systems:
   ```
   sudo apt-get update
   sudo apt-get install --only-upgrade openssl
   ```

For Red Hat/CentOS Systems:

```
sudo yum update openssl
```

After updating OpenSSL, restart the Apache web server to ensure the changes take effect:

```
sudo systemctl restart apache2   # For Debian/Ubuntu
sudo systemctl restart httpd     # For Red Hat/CentOS
```

4. **Regenerating SSL Certificates (If Necessary):**
   If the server was exposed to Heartbleed, assume that SSL/TLS private keys may have been compromised. In such cases, revoke the old certificates and generate new ones:

   ```
   openssl genrsa -out new_private_key.pem 2048
   openssl req -new -key new_private_key.pem -out new_cert_request.csr
   ```

   Then, obtain a new certificate from the Certificate Authority (CA) and update the Apache SSL configuration accordingly.
5. **Final Validation:**
   After applying the patches, verify that the server is no longer vulnerable:

   ```
   openssl version -a
   nmap --script ssl-heartbleed -p 443 <server_ip>
   ```

   If the vulnerability is no longer detected, the system is secure.